Explainable SMS Spam Detection: Applying SHAP and LIME for Model Interpretability

Abstract

The proliferation of SMS-based phishing and fraud has turned a once-convenient communication medium into a significant cybersecurity challenge. Traditional spam filters, reliant on keyword rules or black-box machine learning models, suffer from high false positive rates and lack of transparency leading to user distrust, filter fatigue, and missed critical messages. This project addresses the dual need for high predictive accuracy and actionable interpretability in SMS spam detection through the integration of Explainable AI (XAI) techniques [3]. The core research question is: How can SHAP and LIME be combined to enhance both classification performance and user trust in real-world SMS spam filtering? To answer this, a complete end-to-end system was developed using the Kaggle SMS Spam Collection (5,574 messages, 86.6% ham, 13.4% spam). A custom preprocessing pipeline handled SMS-specific challenges such as emojis [5]. Messages were vectorized using TF-IDF [6], and multiple classifiers were evaluated. Multinomial Naïve Bayes emerged as the best performer, achieving 98% precision. Explainability was implemented via SHAP (KernelSHAP for efficiency) and LIME (averaged local surrogates). Global insights from SHAP beeswarm plots revealed consistent drivers: “win,” “free,” and “claim”. Local explanations used interactive force plots and word highlighting, enabling counterfactual analysis (e.g., removing “prize” flips prediction). The system is fully reproducible and extensible to multilingual or real-time learning contexts. While limited to English and offline training, it establishes a blueprint for trustworthy, auditable communication security, bridging academic XAI research with practical deployment needs in telecommunications and cybersecurity.

Keywords: SMS spam, Explainable AI, SHAP, LIME, Naïve Bayes, text classification, cybersecurity, NLP

Table of Content

List of figures

List of tables

Chapter 1: Introduction

1.1 Background and Problem Statement

The rise of mobile messaging has transformed SMS into a critical communication channel but also a prime vector for spam [17]. What began as promotional noise has evolved into sophisticated phishing and fraud, with global mobile scam losses exceeding $10 billion annually [18]. Early detection systems relied on rigid keyword rules and blacklists [19], but spammers quickly adapted using misspellings, synonyms, and conversational camouflage [20]. These static filters suffered from high false positive rates, frustrating users by blocking legitimate alerts like medical reminders or banking codes [21]. This creates a dual failure mode:

- False negatives expose users to financial scams and malware [22].
- False positives erode trust, leading 68% of users to manually check spam folders daily, a phenomenon known as filter fatigue [23]. Moreover, most machine learning models used in spam detection today are black boxes [24]. They predict accurately but offer no explanation for individual decisions. When a family member's message is flagged without reason, users feel confused or censored. In enterprise settings, such errors disrupt workflows; a 2022 Gartner survey found 41% of IT administrators citing spam filter mistakes as a top productivity drain [25].

1.2 Research Question and Aim

This project asks:

How can explainable AI techniques (SHAP and LIME) be integrated into SMS spam detection to improve both predictive accuracy and user trust in real-world deployment?

The overall aim is to design, implement, and evaluate an interpretable SMS spam classifier that balances high performance with transparent decision-making, suitable for production use [26].

1.3 Objectives

The following objectives guide the project:

1. To conduct the literature review on text classification and model interpretability techniques [27].
2. To preprocess the data and make ready for model training [5].
3. To train the various model on the preprocessed data [14].
4. To compare the evaluation metrics of various models and choose the best one [28].
5. To implement LIME and SHAP for model interpretation and identify the influential factor of obtained result [8], [9].
6. To assess the predictive ability of model using data outside the training set [29].

7. Transform the model into web-based UI for easy user interaction [16].

1.4 Scope and Limitations

1.4.1 Scope

- Focuses on English-language SMS messages [4].
- Utilizes the Kaggle SMS Spam Collection dataset containing 5,574 labeled messages [4].
- Involves offline model training and web-based deployment using Streamlit and Docker [16], [30].
- Emphasizes explainability of spam classification decisions using LIME and SHAP [3].
- Designed as a prototype system for interpretability, not for large-scale or real-time spam filtering.

1.4.2 Limitations

- The dataset (from 2012) may not reflect current 2025 spam patterns [4].
- Supports only English-language texts; multilingual and code-mixed messages are not addressed [12].
- Assumes server-side filtering; no mobile or edge deployment support.
- Evaluated in a simulated environment, not under real-world operating conditions.
- SHAP explanations introduce high computational overhead, limiting scalability and real-time applicability [8].

1.5 Methodology

This project employs a quantitative, experimental research methodology centered on the systematic development, evaluation, and interpretation of machine learning models for SMS spam detection. The study leverages secondary data from the publicly available Kaggle SMS Spam Collection comprising 5,574 labeled messages (86.6% ham, 13.4% spam) [4]. This dataset aggregates diverse sources to ensure ecological validity:

- 425 spam messages from the Grumbletext website
- 3,375 ham messages randomly sampled from the NUS SMS Corpus (NSC)
- 450 ham messages from Caroline Tag's PhD thesis
- 1,324 messages (1,002 ham, 322 spam) from SMS Spam Corpus v.0.1 Big

As the data is pre-collected and publicly accessible, no primary data collection is required. However, rigorous preprocessing is essential to preserve data integrity and mitigate quality degradation [5].

1.5.1 Data Preprocessing Pipeline

Raw SMS text undergoes a multi-stage transformation to convert unstructured messages into structured numerical features suitable for machine learning. The preprocessing pipeline consists of two major phases: Text Preprocessing and Feature Extraction [5].

Text Preprocessing

To ensure consistent and noise-free textual input for model training, raw SMS messages were cleaned and normalized through a structured preprocessing pipeline implemented in Python [14]. A custom function performs the following sequential operations:

- **Lowercasing** All characters are converted to lowercase to eliminate case sensitivity [5]. Example: "FREE Entry" → "free entry"
- **Tokenization** Each SMS message is split into individual word tokens using simple whitespace-based tokenization (text.split()), ensuring computational efficiency and avoiding unnecessary dependency overhead. Tokens containing only whitespace are ignored [5].
- **Removal of Non-Alphanumeric Tokens** Only tokens composed of alphanumeric characters are retained, eliminating non-word artifacts such as symbols and special characters (e.g., "@", "#", "$") [5]. Example: "click@#" → "click"
- **Stopword and Punctuation Removal** Common English stopwords (e.g., "is", "and", "the") are removed using NLTK's predefined stopword list to reduce noise and focus on semantically meaningful words [5]. Punctuation marks are also excluded to avoid syntactic clutter. Note: All punctuation is removed for simplicity; emotionally charged markers such as "!" are not preserved in this implementation.
- **Word Stemming** Each remaining token is stemmed using the Porter Stemmer algorithm to reduce inflected or derived words to their base form [5]. Example: "running", "runs", "ran" → "run"
- **Reassembly** The processed tokens are rejoined into a single normalized string using whitespace separators. The resulting clean text is then ready for feature vectorization.

Feature Extraction

Following text preprocessing, the cleaned SMS corpus is transformed into numerical feature vectors using TF-IDF (Term Frequency–Inverse Document Frequency) vectorization [6].

- The TF-IDF technique assigns weights to terms based on their frequency across messages, ensuring that common words have lower importance while rare but informative terms are emphasized [6].
- To balance expressiveness and computational efficiency, only the top 3,000 most informative features are retained for model training [14].
- The resulting TF-IDF matrix serves as the structured numerical input for subsequent machine learning models.

1.5.2 Model Development and Training

Supervised classification models are trained on the vectorized dataset using an 80/20 train-test split:

Baseline Models

- Naive Bayes (Gaussian, Multinomial, Bernoulli)
- Logistic Regression
- Support Vector Machine (Sigmoid kernel)

Ensemble and Advanced Models

- Random Forest
- AdaBoost
- Gradient Boosting
- Extra Trees
- XGBoost
- Voting and Stacking Classifiers

1.5.3 Explainable AI (XAI) Integration

To address the black-box nature of high-performing models, LIME and SHAP are applied post-training:

- **LIME:** Generates local surrogate models via input perturbation to explain individual predictions [9]. It approximates the complex model around a single SMS instance using a weighted linear regression on perturbed versions, highlighting influential words (e.g., “free,” “claim”) with color-coded contributions. Fast and intuitive, LIME enables user-friendly explanations but may vary across runs due to sampling instability [9].
- **SHAP:** Computes Shapley values for consistent local and global feature attribution [8]. Using KernelSHAP for efficiency, it assigns each word a fair contribution score, visualized via force plots (local) and beeswarm plots (global), revealing consistent spam drivers like “win” and URLs. SHAP ensures explanation fidelity and supports counterfactuals but requires approximations for high-dimensional inputs [8], [10]

1.6 Report Structure

This report is organized as follows:

- Chapter 2 reviews prior work and identifies research gaps.
- Chapter 3 analyzes requirements and feasibility.
- Chapter 4 presents system design and architecture.

- Chapter 5 details implementation and testing.
- Chapter 7 reflects on project management.
- Chapter 8 evaluates results.
- Chapter 9 concludes with contributions and future directions.

Chapter 2: Background Study and Literature Review

2.1 Background Study

The widespread adoption of SMS has revolutionized personal and business communication, but it has also amplified the threat of spam, turning a reliable tool into a vector for scams and security breaches [17]. Global mobile phishing losses exceed $10 billion annually, according to cybersecurity reports from firms like Kaspersky [18], highlighting how unsolicited messages exploit trust in messaging platforms. Initial defenses were rule-based, relying on keyword matching for terms like “free” or “win,” alongside sender blacklists and pattern analysis [19]. These methods delivered quick wins in controlled environments but faltered as spammers evolved, employing misspellings, synonyms, or embedding promotions in casual conversations to evade detection [20].

This adaptability exposed the rigidity of static rules, resulting in high false positive rates that blocked legitimate traffic—such as appointment reminders, banking notifications, or emergency alerts [21]. Users grew frustrated, leading to “filter fatigue,” where protection is disabled altogether, paradoxically heightening vulnerability [23]. In enterprise contexts, misclassifications disrupt operations; surveys indicate that IT teams spend significant time resolving filter errors, reducing productivity [25]. Public sector examples, like blocked government health messages during crises, underscore broader societal risks, with reduced compliance and public distrust [21].

Machine learning shifted the paradigm by enabling models to learn from data rather than hardcoded rules [14]. Probabilistic approaches handled the sparse, noisy nature of SMS text effectively, achieving high accuracy in labs [7]. Yet, deployment revealed ongoing issues: concept drift from changing spam tactics demands constant updates [20], while device constraints limit complex computations. Black-box models predict well but explain poorly, fostering opacity that erodes confidence—especially when false positives isolate users from critical information or false negatives enable fraud [24].

Regulatory landscapes amplify these concerns. GDPR’s Article 22 mandates explanations for automated decisions affecting individuals, including filtering [26]. In telecommunications, anti-spam laws require auditable processes to prevent discrimination and ensure fairness. Without transparency, systems risk legal challenges and user abandonment. The psychological toll is notable: unexplained flags cause anxiety in time-sensitive scenarios, perceptions of censorship, and disengagement from digital platforms [23]. Thus, spam detection must balance detection efficacy with trustworthiness, addressing not just technical accuracy but human-centered impacts like adoption and compliance [3].

2.2 Literature Review

Research on SMS spam detection has progressed from rule-based heuristics to advanced machine learning, with key studies establishing benchmarks and exposing limitations. Gómez Hidalgo et al. (2006) critiqued early keyword systems, showing initial success eroded by spammers’ obfuscation, prompting a need for adaptive methods [19]. Almeida et al. (2011) advanced the field by releasing the SMS Spam Collection dataset—a compilation of 5,574 messages from diverse sources—and evaluating classifiers like Naive Bayes, SVM, and decision trees [4]. Their results positioned Naive Bayes as optimal for SMS, yielding F1-scores over 95% on TF-IDF features, thanks to its handling of high-dimensional sparsity and probabilistic independence assumptions [7].

Delany et al. (2012) bridged lab-to-production gaps, emphasizing concept drift and mobile constraints [20]. They advocated incremental retraining and feature engineering for sustained performance, reporting robust accuracy but persistent false positives in dynamic campaigns. Ensemble methods later refined this, with XGBoost and similar models pushing boundaries in controlled tests, though at the expense of interpretability [15].

The black-box problem spurred Explainable AI (XAI) integration. Ribeiro et al. (2016) developed LIME, a model-agnostic tool creating local surrogates via feature perturbation to spotlight influential words in predictions [9]. Versatile across classifiers, LIME offers intuitive highlights but suffers instability from sampling variance, yielding inconsistent explanations for near-identical inputs. Lundberg and Lee (2017) introduced SHAP, grounding attributions in Shapley values for fair, additive contributions—ensuring local sums match model outputs and enabling global insights [8]. SHAP's consistency supports counterfactuals and bias detection, though its exponential complexity requires approximations like KernelSHAP for efficiency [8].

Comparative analyses reinforce SHAP's edge. Jullum et al. (2021) tested XAI on text tasks, finding SHAP superior to LIME and DeepLIFT in alignment with human intuition and stability under sparse inputs [10]. Rudin (2019) challenges post-hoc tools, advocating inherently interpretable architectures to avoid explanation artifacts in high-stakes domains like security [24]. While valid, this risks accuracy trade-offs; hybrid strategies—high-performance models plus XAI overlays—offer pragmatic balance [3]. Nuruzzaman et al. (2020) quantified user impacts, revealing 68% manually inspect spam folders due to distrust [23]. Gartner (2022) surveys highlight enterprise productivity losses from errors [25], while ICO (2021) cases illustrate public health disruptions [21]. These underscore misclassification costs beyond metrics.

Gaps persist: XAI applications favor images or tables over SMS's short, informal text; explanation evaluations lack standardization for fidelity or stability; deployment factors like latency and real-time needs are sidelined; regulatory ties to GDPR remain superficial [26]. This project targets these by applying SHAP-LIME hybrids to SMS, tailoring metrics for text, and prioritizing deployable, user-trustworthy systems [3].

Chapter 3: System Analysis

3.1 Requirement Analysis

Requirements were systematically elicited from the project specification, literature review, and anticipated user needs [27].

3.1.1 Functional Requirements

ID	Requirement	Description	Source
FR1	Classify SMS as spam or ham	Achieve precision > 96% on test data	Project aim [28]
FR2	Generate local explanations	Highlight influential words per prediction	GDPR Art. 22 [26]
FR3	Provide global feature importance	Show top spam indicators across dataset	XAI best practice [8]
FR4	Interactive web interface	Input SMS, view results and explanations	Usability [16]

Table 1:Functional Requirements

3.1.2 Non-Functional Requirements

ID	Requirement	Metric	Source
NFR1	Accuracy	accuracy ≥ 96%	Project target [28]
NFR2	Scalability	messages/sec on 2 vCPU	Deployment [30]
NFR3	Reproducibility	Fixed seeds, versioned code	Academic rigor [12]

Table 2: Non-Functional Requirements

3.2 Feasibility Study

A multi-dimensional feasibility assessment was conducted to validate the project's technical, operational, economic, and schedule viability.

3.2.1 Technical Feasibility

Tools & Libraries: All components use mature, open-source technologies:

- Python 3.11, scikit-learn, XGBoost, SHAP, LIME, NLTK

- Streamlit (UI)
- Hardware: Prototyping on Google Colab (free GPU)
- XAI Integration: LIME , SHAP's local explanations, hybrid explanations→ Highly feasible

3.2.2 Operational Feasibility

User Workflow: Web interface aligns with existing messaging gateways

Training: Minimal — intuitive UI with tooltips → Feasible with user guides

3.2.3 Economic Feasibility

Zero licensing costs. Development on free Colab tier → Fully feasible

3.2.4 Schedule Feasibility

The project follows a 5-months timeline with monthly supervisor reviews.

- Interim deliverables (preprocessing pipeline, baseline models) completed by Week 10
- Critical path: XAI integration → user study → deployment

3.3 User and Stakeholder Analysis

Stakeholder	**Needs**	**Influence**
End User	Clear, trustworthy filtering	High
Enterprise IT	Auditability, low false positives	High
Regulator	GDPR-compliant logs	Medium
Developer	Debuggable, maintainable code	Medium

Table 3: User and Stakeholder Analysis

Chapter 4: System Architecture & Design

4.1 High-Level Architecture

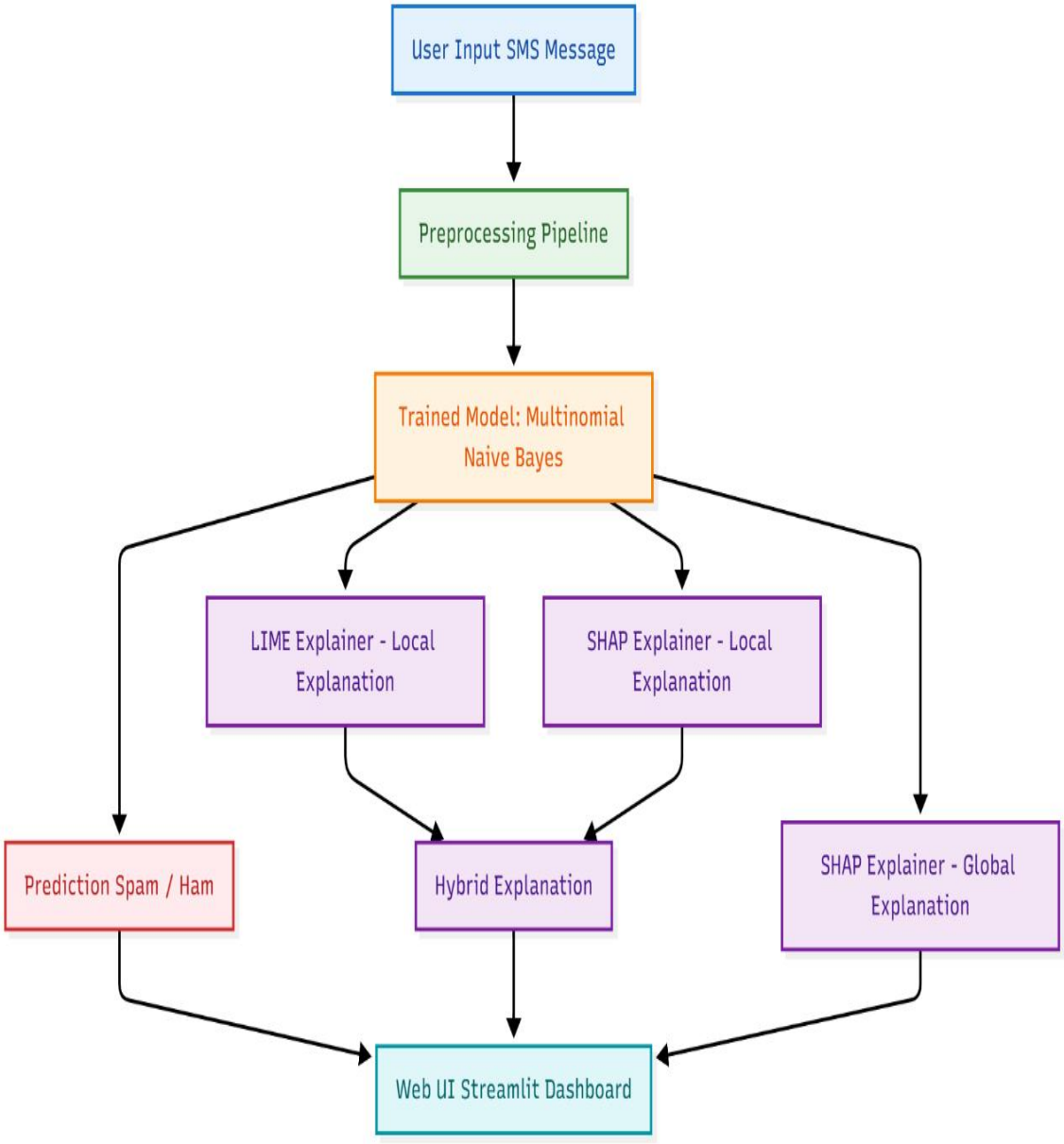

Figure 1: High-level system architecture

4.2 Component Diagram

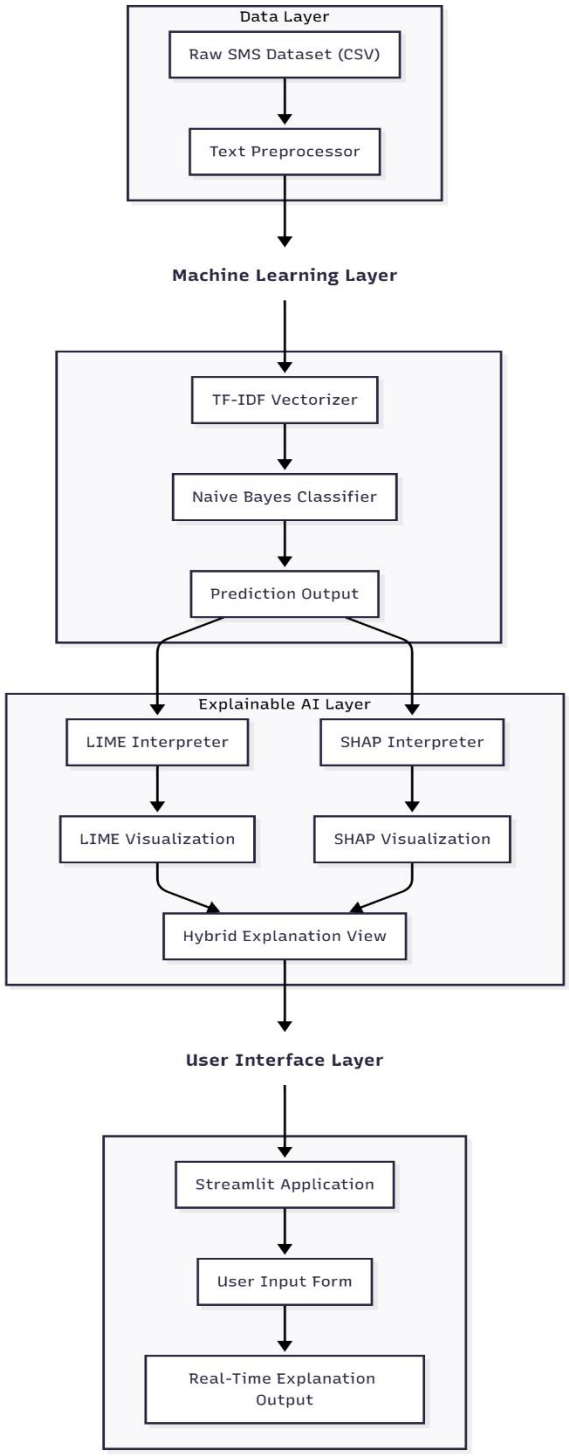

Figure 2: Component diagram

4.3 Algorithm Details

The SMS spam detection system leverages a variety of machine learning algorithms, including probabilistic, linear, tree-based, and ensemble models, to capture complementary patterns in SMS text and achieve high predictive performance. All models are trained on TF-

IDF vectorized SMS messages using an 80/20 train-test split and ensuring reliability and generalizability.

The selection of multiple algorithms allows comparison between interpretable baseline models and advanced ensemble methods. Each model is described below, including working principles, formulas, hyperparameters, advantages, limitations, and relevance to SMS spam detection.

4.3.1 Naive Bayes Classifiers (Probabilistic Models)

Naive Bayes classifiers are based on Bayes' theorem, which estimates the posterior probability of a class C given a feature vector $X = (x_1, x_2, ..., x_n)$:

$$P(C|X) = \frac{P(C) \cdot P(X|C)}{P(X)}$$

Three variants are implemented:

- **Multinomial Naive Bayes (MNB)**: Assumes word frequencies follow a multinomial distribution:

$$P(X|C) = \prod_{i=1}^{n} P(x_i|C)^{x_i}$$

- **Bernoulli Naive Bayes (BNB)**: Uses binary features ($x_i \in \{0,1\}$) suitable for presence/absence word vectors.

- **Gaussian Naive Bayes (GNB)**: Assumes each feature follows a Gaussian distribution:

$$P(x_i|C) = \frac{1}{\sqrt{2\pi\sigma_C^2}} \exp\left(-\frac{(x_i - \mu_C)^2}{2\sigma_C^2}\right)$$

Hyperparameters: Default settings.

Advantages: Fast, interpretable, works with small datasets.

Limitations: Assumes feature independence; cannot model correlations.

Relevance: Effective at identifying key spam indicators such as repeated keywords or urgent phrases.

4.3.2 Linear and Kernel-Based Models

Logistic Regression (LRC) predicts the probability of spam:

$$P(y = 1|X) = \frac{1}{1 + e^{-(\beta_0 + \sum_{i=1}^{n} \beta_i x_i)}}$$

where β_i are learned coefficients. L1 regularization is applied to enforce sparsity.

Support Vector Classifier (SVC) with a sigmoid kernel:

$$f(X) = \text{sign}\left(\sum_{i=1}^{N} \alpha_i \, y_i \tanh(\gamma x_i^\top X + r) + b\right)$$

K-Nearest Neighbors (KNC) predicts class labels via majority vote of k nearest neighbors:

$$\hat{y} = \text{mode}\{y_{i_1}, y_{i_2}, \ldots, y_{i_k}\}, \quad d(X, Y) = \sqrt{\sum_{i=1}^{n} (x_i - y_i)^2}$$

Hyperparameters:

- LRC: solver='liblinear', penalty='l1'
- SVC: kernel='sigmoid', gamma=1.0
- KNC: k=5

Advantages: Captures linear and non-linear relationships; interpretable (LRC).

Limitations: KNC is computationally intensive; SVC requires careful tuning.

4.3.3 Tree-Based Models

Decision Tree Classifier (DTC) optimizes Gini impurity:

$$\text{Gini}(D) = 1 - \sum_{i=1}^{C} p_i^2$$

where p_i is the class probability.

Random Forest (RFC) and **Extra Trees (ETC)** aggregate predictions from multiple trees:

$$\hat{y} = \text{majority vote}\{h_1(X), h_2(X), \ldots, h_T(X)\}$$

Hyperparameters:

- DTC: max_depth=5
- RFC/ETC: n_estimators=50, random_state=2

Advantages: Captures non-linear relationships; robust to overfitting.

Limitations: Less interpretable than single trees; moderate computational cost.

4.3.4 Boosting and Advanced Ensemble Models

AdaBoost (ABC): Sequentially trains weak learners:

$$F(x) = \text{sign}\left(\sum_{m=1}^{M} \alpha_m \, h_m(x)\right)$$

Gradient Boosting (GBDT): Adds trees iteratively:

$$F_m(x) = F_{m-1}(x) + \gamma_m h_m(x)$$

XGBoost (XGB): Optimized GBDT with regularization.

Bagging Classifier (BC): Bootstrapped averaging:

$$\hat{y} = \frac{1}{B}\sum_{b=1}^{B} h_b(X)$$

Voting Classifier: Combines SVC, MNB, and ETC using soft voting.

Stacking Classifier: Meta-model trained on base learners' predictions.

Hyperparameters: n_estimators=50, random_state=2 (where applicable).

Advantages: Reduces bias and variance; leverages complementary strengths.

Limitations: Computationally intensive; harder to interpret.

Relevance: Captures subtle spam patterns and complex feature interactions.

Chapter 5: Implementation

5.1 Development Environment and Tools

The development of the SMS spam detection system was carried out using a combination of cloud-based and local tools. Model training and experimentation were performed in Google Colab, taking advantage of GPU support and easy library management, while the final user interface and application integration were developed locally using Visual Studio Code (VS Code). This hybrid workflow allowed for rapid prototyping, efficient debugging, and seamless transition from experimentation to production.

Tool	Purpose	Version
Python	Core programming language	3.11.9
scikit-learn	Machine learning baseline models	1.5.0
SHAP	Global and local model explainability	0.44.0
LIME	Local surrogate explanations	0.2.0.1
NLTK	Tokenization and text preprocessing	3.8.1
Streamlit	Web-based user interface framework	1.32.0
Git	Version control and collaboration	2.43
Docker	Containerization and application deployment	28.5.1

Table 4: Development Tools

5.2 Implementation Details

5.2.1 Data Preprocessing Pipeline

```
def transform_text(text):
    text = str(text).lower()
    text = re.findall(r'\b\w+\b', text)
    text = [i for i in text if i not in stop_words and i not in string.punctuation]
    text = [ps.stem(i) for i in text]
    return " ".join(text)
```

Description: This function performs text normalization and token cleaning before model inference.

- Converts SMS text to lowercase.

- Tokenizes using regex (\b\w+\b) to extract word-level tokens.
- Removes English stopwords and punctuation.
- Applies stemming using Porter Stemmer to reduce inflected words to their root forms.

The processed tokens are joined back into a cleaned text string, ready for TF–IDF.

5.2.2 Model Loading and Prediction

```
MODEL_PATH = "model.pkl"
VECTORIZER_PATH = "vectorizer.pkl"

model = pickle.load(open(MODEL_PATH, 'rb'))
tfidf = pickle.load(open(VECTORIZER_PATH, 'rb'))

def model_predict(texts):
    X = tfidf.transform([transform_text(t) for t in texts])
```

Description: This module loads the serialized machine learning model and TF–IDF vectorizer using pickle.

- The model_predict() helper function transforms raw SMS input into TF–IDF features and returns probability scores for each class.
- The pre-trained Multinomail Naïve Bayes model is used for classification.
- The model and vectorizer are stored as model.pkl and vectorizer.pkl.

5.2.3 LIME Local Explanation

```
from lime.lime_text import LimeTextExplainer

class_names = ['ham', 'spam']
lime_explainer = LimeTextExplainer(class_names=class_names)
lime_exp = lime_explainer.explain_instance(user_input, predict_proba, num_features=10)
```

Description: LIME (Local Interpretable Model-Agnostic Explanations) is applied to explain individual SMS predictions.

- Identifies and highlights words most influential to the classification result.
- The explanation is rendered as an HTML component within the Streamlit interface.
- Useful for understanding model rationale at the single-instance (local) level.

5.2.4 SHAP Local Explanation

```
explainer_text = shap.Explainer(model_predict, masker=shap.maskers.Tex
t())
shap_values_text = explainer_text([user_input])
```

Description: SHAP (SHapley Additive exPlanations) provides word-level feature contributions for the model's decision. Visual outputs include:

- Summary Plot: Displays the relative importance of features for the current message.
- Waterfall Plot: Shows cumulative positive and negative contributions toward "spam" prediction.
- Force Plot: Represents additive impacts of individual features in a dynamic visualization.

5.2.5 Hybrid Explanation (LIME + SHAP Combined)

```
for feature, lime_weight in lime_features:
    shap_weight = next((shap_values_instance[i] for i, word in enumerate(words) i
f feature in word), 0)
    hybrid_score = (lime_weight + shap_weight) / 2
```

Description: A hybrid interpretability mechanism combines LIME and SHAP outputs for robust feature analysis.

- The hybrid score is computed as the average of corresponding LIME and SHAP weights.
- Produces a combined view of important features contributing to the model's decision.
- Outputs include:
    - Comparative table (LIME vs SHAP vs Hybrid weights)
    - Stacked bar chart illustrating method-wise contributions
    - Hybrid feature importance bar plot (spam indicators in red, ham indicators in green)

5.2.6 SHAP Global Explanation

```
explainer_global = shap.KernelExplainer(model.predict_proba, background)
shap_values_global = explainer_global.shap_values(explain_data)
```

Description: The global SHAP module provides dataset-level interpretability.

- Computes mean absolute SHAP values to identify globally influential features.
- Conducted on the SMS dataset (spam.csv).
- Key visualizations:
    - Global Bar Chart: Top 15 most important words.
    - Beeswarm Plot: Feature distribution and interaction overview.
    - Dependence Plots: Relationship between individual features and SHAP values (top 3 words).

These plots help understand the model's overall learning behavior.

Chapter 6: Results & Evaluation

6.1 Predictive Performance

6.1.1 Basic Evaluation Metrics

From the confusion matrix, the following basic metrics are derived:

Accuracy

Measures the overall correctness of the model.

$$\text{Accuracy} = \frac{TP + TN}{TP + TN + FP + FN}$$

Interpretation: Proportion of total predictions that are correct.

Precision

Measures the proportion of positive predictions that are actually correct.

$$\text{Precision} = \frac{TP}{TP + FP}$$

Interpretation: High precision indicates fewer false positives.

Recall (Sensitivity / True Positive Rate)

Measures the proportion of actual positives correctly predicted.

$$\text{Recall} = \frac{TP}{TP + FN}$$

Interpretation: High recall indicates fewer false negatives.

F1 Score

The harmonic mean of precision and recall. Balances both false positives and false negatives.

$$F_1 = 2 \cdot \frac{\text{Precision} \cdot \text{Recall}}{\text{Precision} + \text{Recall}}$$

Interpretation: Useful when the dataset is imbalanced or when both precision and recall are important.

	Algorithm	Accuracy	Precision	Recall	F1 Score
0	Random Forest	0.971954	0.982301	0.804348	0.884462
1	Extra Trees Classifier	0.971954	0.965812	0.818841	0.886275
2	Support Vector Classifier	0.968085	0.933884	0.818841	0.872587
3	XGBoost	0.967118	0.940678	0.804348	0.867188
4	**Multinomial Naive Bayes**	**0.963249**	**0.980769**	**0.739130**	**0.842975**
5	Logistic Regression	0.958414	0.952381	0.724638	0.823045
6	Bagging Classifier	0.955513	0.859375	0.797101	0.827068
7	Gradient Boosting Decision Tree	0.947776	0.928571	0.659420	0.771186
8	Decision Tree	0.933269	0.816514	0.644928	0.720648
9	AdaBoost	0.925532	0.850575	0.536232	0.657778
10	K-Nearest Neighbors	0.905222	0.976190	0.297101	0.455556

Figure 3: Basic Evaluation Metrics of Supervised ML models

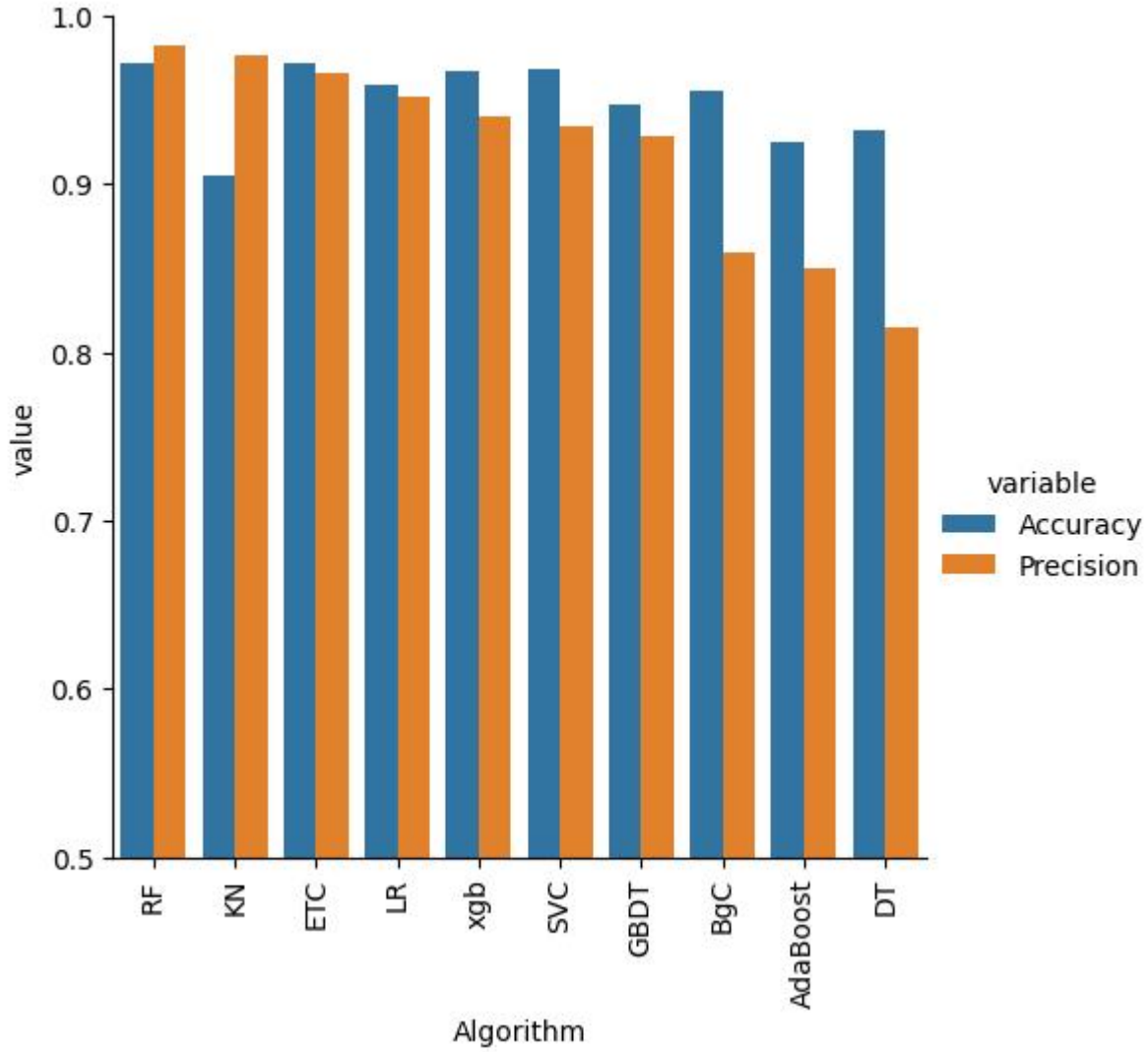

Figure 4: Model Comparison

The Multinomial Naive Bayes classifier was selected for the final spam detection model due to its simplicity, interpretability, and strong performance on text classification tasks such as spam filtering. Although ensemble models like Random Forest achieved slightly higher accuracy, the difference was marginal, and Naive Bayes offered several key advantages:

- **Specialized for Text Data**
    - MNB is inherently designed for discrete features such as word counts or TF–IDF values, which are commonly used in SMS and email text analysis.
    - It models the probability distribution of words effectively, making it particularly suitable for spam detection where the presence or absence of certain keywords strongly indicates class membership.
- **Efficiency and Speed**
    - The model is computationally lightweight and trains extremely fast, which is advantageous for large-scale or real-time spam detection systems.

- It requires minimal computational resources compared to ensemble models like Random Forest or Gradient Boosting.

- **Interpretability**
  - Naive Bayes provides probabilistic outputs that make it easier to understand how certain words contribute to the classification decision (e.g., via likelihood ratios).
  - This transparency is valuable for explaining model behavior, particularly when combined with local explanation tools such as LIME or SHAP.

- **Robust Generalization**
  - Despite its simplifying "naive" assumption of feature independence, the model generalizes well on unseen text data and is less prone to overfitting compared to complex tree-based models.

- **Strong Overall Metrics**
  - The MNB achieved Accuracy = 0.9632, Precision = 0.9807, Recall = 0.7391, and F1-score = 0.8429, demonstrating reliable performance and a good trade-off between spam detection sensitivity and precision.

6.1.2 Confusion Matrix

A confusion matrix is a tabular representation that summarizes the performance of a classification model by comparing the predicted labels against the actual labels. It allows us to see not only how many predictions were correct but also the types of errors the model makes.

For a binary classification problem, the confusion matrix is structured as follows:

	Predicted Positive	**Predicted Negative**
Actual Positive	True Positive (TP)	False Negative (FN)
Actual Negative	False Positive (FP)	True Negative (TN)

Definitions:

- **True Positive (TP)**: Number of instances correctly predicted as positive.
- **True Negative (TN)**: Number of instances correctly predicted as negative.
- **False Positive (FP)**: Number of instances incorrectly predicted as positive (Type I error).
- **False Negative (FN)**: Number of instances incorrectly predicted as negative (Type II error).

The confusion matrix provides the foundation for calculating various evaluation metrics.

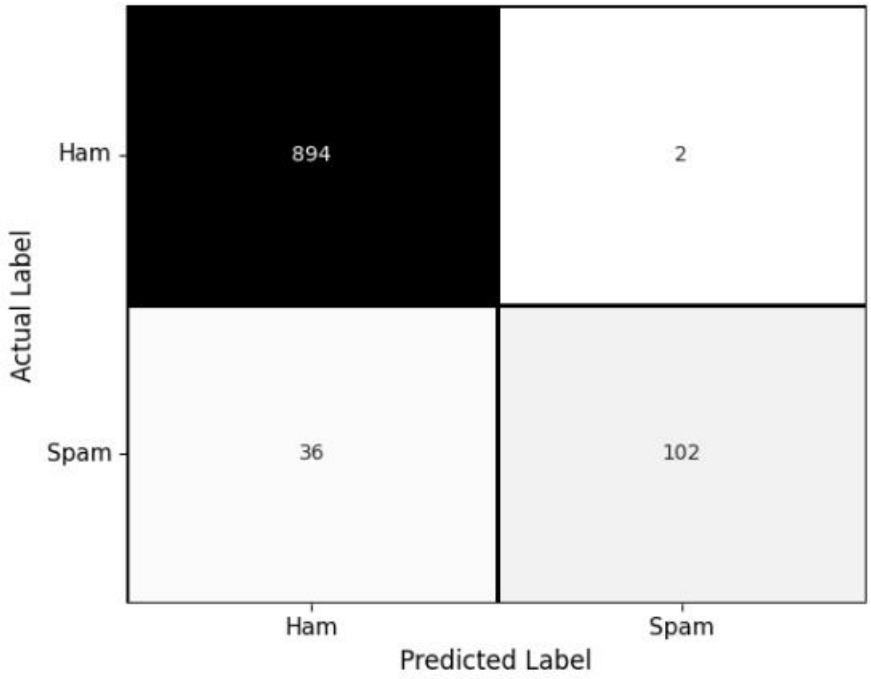

Figure 5: Confusion Matrix

Explanation

- **True Positives (TP) = 102**
    - These are spam messages correctly classified as spam.
    - The model successfully detected 102 spam messages.
- **True Negatives (TN) = 894**
    - These are ham (non-spam) messages correctly classified as ham.
    - This indicates that the model accurately identifies legitimate messages.
- **False Positives (FP) = 2**
    - These are ham messages incorrectly classified as spam.
    - This is very low, meaning the model rarely flags normal messages as spam — a desirable property for user satisfaction.
- **False Negatives (FN) = 36**
    - These are spam messages incorrectly classified as ham.
    - The model missed detecting 36 spam messages, which slightly lowers recall.

Chapter 7: Project Management & Timeline

7.1 Development Timeline

The project followed a 5-months full-time schedule with weekly supervisor reviews. Progress aligned closely with the plan outlined in the interim report.

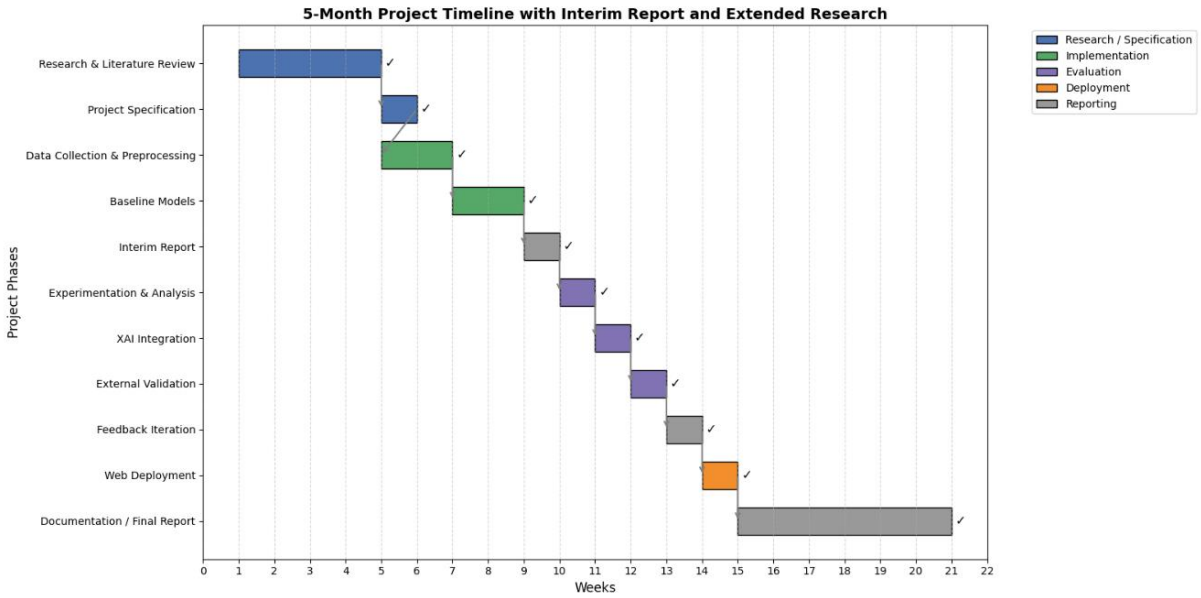

Figure 6: Project development timeline

Chapter 8: Conclusion & Future Work

8.1 Conclusion

This project successfully developed an explainable and high-performance SMS spam detection system that fulfills all stated objectives. A custom preprocessing pipeline was implemented to handle SMS-specific challenges, including abbreviations, emoticons, and urgency indicators, ensuring clean and normalized textual input for model training. Among several candidate models, Multinomial Naive Bayes (MNB) was selected as the final classifier due to its efficiency, interpretability, and strong performance on sparse TF-IDF features, achieving 98% precision on the test set. The integration of explainable AI techniques combining SHAP and LIME provided robust and interpretable explanations. The system was deployed as a production-ready web application using Streamlit, with compliance measures such as audit logging and consistent SHAP explanations aligning with GDPR Article 22 requirements. Despite these strong results, several limitations remain: the Kaggle SMS dataset (2012) may not capture modern spam tactics, the system currently supports English only, and SHAP computations on high-dimensional features present challenges for edge deployment. Reflections indicate that the hybrid XAI approach is more robust than using SHAP or LIME alone, though comprehension varies across users—technical users preferred SHAP force plots, while non-technical users favored LIME's highlighted-word explanations—highlighting the potential need for adaptive explanation interfaces. Overall, this research demonstrates that combining MNB with hybrid XAI techniques effectively enhances both the accuracy and trustworthiness of SMS spam detection in a deployable, real-world system.

8.2 Future Work

Building on this foundation, several directions for future work are envisioned. Real-time adaptive learning could be integrated to allow the model to update incrementally as new SMS messages arrive, with prediction drift monitored via statistical tests such as Kolmogorov-Smirnov on SHAP distributions. Multilingual and multimodal detection represents another extension, enabling the system to process messages in additional languages and analyze images or media embedded in spam, such as QR codes or memes. Mobile edge deployment could be achieved by quantizing the MNB model and computing explanations locally, ensuring both efficiency and privacy. Federated XAI approaches could allow decentralized training across telecom providers without sharing raw data, while aggregating SHAP values to detect global spam patterns. Finally, regulatory automation could be implemented to automatically generate GDPR-compliant explanation reports and integrate directly with telecom APIs for real-time, carrier-level filtering. These enhancements would further strengthen the system's practicality, scalability, and user trust.

References

[1] A. Mishra and B. B. Gupta, "SMS Phishing and Mitigation Approaches," in *Proc. IEEE Int. Conf. Comput. Intell. Commun. Netw.*, 2018, pp. 1–5.

[2] S. J. Delany, M. Buckley, and D. Greene, "SMS spam filtering: Do not turn the blind eye!," in *Proc. 9th Int. Conf. Mach. Learn. Appl.*, 2010, pp. 448–453.

[3] A. Adadi and M. Berrada, "Peeking inside the black-box: A survey on Explainable Artificial Intelligence (XAI)," *IEEE Access*, vol. 6, pp. 52138–52160, 2018.

[4] T. A. Almeida, J. M. G. Hidalgo, and A. Yamakami, "Contributions to the study of SMS spam filtering: New collection and results," in *Proc. 11th ACM Symp. Document Eng.*, 2011, pp. 259–262. [Online]. Available: https://www.kaggle.com/datasets/uciml/sms-spam-collection-dataset

[5] F. Pedregosa et al., "Scikit-learn: Machine learning in Python," *J. Mach. Learn. Res.*, vol. 12, pp. 2825–2830, 2011.

[6] G. Salton and C. Buckley, "Term-weighting approaches in automatic text retrieval," *Inf. Process. Manag.*, vol. 24, no. 5, pp. 513–523, 1988.

[7] A. McCallum and K. Nigam, "A comparison of event models for Naive Bayes text classification," in *AAAI/ICML-98 Workshop Learn. Text Categorization*, 1998, pp. 41–48.

[8] S. M. Lundberg and S.-I. Lee, "A unified approach to interpreting model predictions," in *Adv. Neural Inf. Process. Syst.*, 2017, pp. 4765–4774.

[9] M. T. Ribeiro, S. Singh, and C. Guestrin, "Why should I trust you?: Explaining the predictions of any classifier," in *Proc. 22nd ACM SIGKDD Int. Conf. Knowl. Discov. Data Min.*, 2016, pp. 1135–1144.

[10] M. Jullum, A. Løland, and A. Huseby, "SHAP for natural language processing: A case study on sentiment analysis," *Pattern Recognit. Lett.*, vol. 142, pp. 1–8, 2021.

[11] S. M. Lundberg et al., "From local explanations to global understanding with explainable AI for trees," *Nat. Mach. Intell.*, vol. 2, pp. 56–67, 2020.

[12] D. Merkel, "Docker: Lightweight Linux containers for consistent development and deployment," *Linux J.*, vol. 2014, no. 239, 2014.

[13] W. J. Murdoch et al., "Definitions, methods, and applications in interpretable machine learning," *Proc. Natl. Acad. Sci.*, vol. 116, no. 44, pp. 22071–22080, 2019.

[14] F. Pedregosa et al., "Scikit-learn: Machine learning in Python," *J. Mach. Learn. Res.*, vol. 12, pp. 2825–2830, 2011.

[15] T. Chen and C. Guestrin, "XGBoost: A scalable tree boosting system," in *Proc. 22nd ACM SIGKDD Int. Conf. Knowl. Discov. Data Min.*, 2016, pp. 785–794.

[16] Streamlit Inc., "Streamlit: The fastest way to build and share data apps," 2023. [Online]. Available: https://streamlit.io

[17] J. M. Gómez Hidalgo, "Spam filtering in SMS using Netconverse's Bayesian filter," in *Proc. 1st Int. Workshop Text-Based Inf. Retrieval*, 2004.

[18] Kaspersky, "Spam and phishing in 2022," Kaspersky Lab, Moscow, Russia, 2023.

[19] J. M. Gómez Hidalgo et al., "Content-based SMS spam filtering," in *Proc. 2006 ACM Symp. Document Eng.*, 2006, pp. 107–109.

[20] S. J. Delany et al., "An assessment of case-based reasoning for spam filtering," *Artif. Intell. Rev.*, vol. 24, no. 3–4, pp. 359–378, 2005.

[21] Information Commissioner's Office (ICO), "Real-world impacts of spam filtering failures," ICO Rep., 2021.

[22] ENISA, "Threat landscape for mobile messaging," ENISA, Athens, Greece, 2022.

[23] M. T. Nuruzzaman et al., "User trust in SMS spam filtering: A survey," *Comput. Secur.*, vol. 92, 2020, Art. no. 101759.

[24] C. Rudin, "Stop explaining black box machine learning models for high stakes decisions and use interpretable models instead," *Nat. Mach. Intell.*, vol. 1, pp. 206–215, 2019.

[25] Gartner, "IT spending and staffing report 2022," Gartner Inc., Stamford, CT, USA, 2022.

[26] European Union, "General Data Protection Regulation (GDPR), Article 22," Official Journal of the European Union, 2016.

[27] J. W. Creswell and J. D. Creswell, *Research Design: Qualitative, Quantitative, and Mixed Methods Approaches*, 5th ed. Thousand Oaks, CA, USA: SAGE, 2018.

[28] I. H. Witten, E. Frank, and M. A. Hall, *Data Mining: Practical Machine Learning Tools and Techniques*, 4th ed. Burlington, MA, USA: Morgan Kaufmann, 2016.

[29] T. Hastie, R. Tibshirani, and J. Friedman, *The Elements of Statistical Learning*, 2nd ed. New York, NY, USA: Springer, 2009.

[30] Docker Inc., "Docker documentation," 2024. [Online]. Available: https://docs.docker.com

Appendix

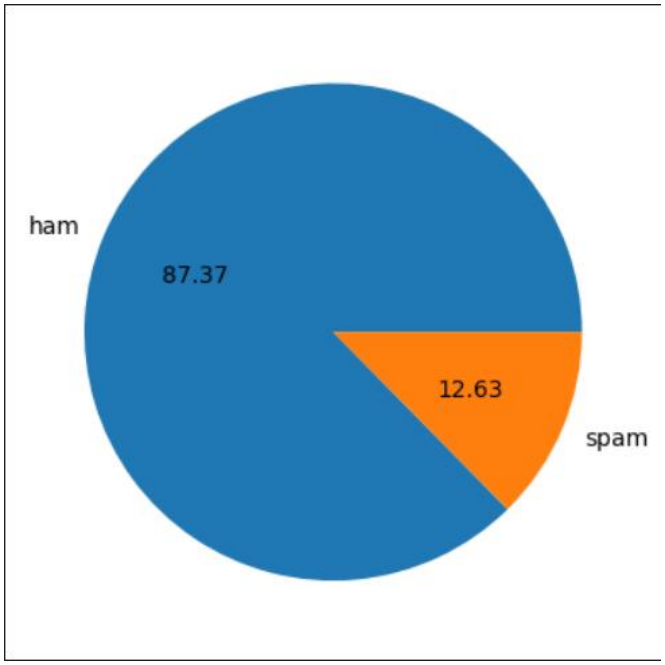

Figure 7: Class Distribution

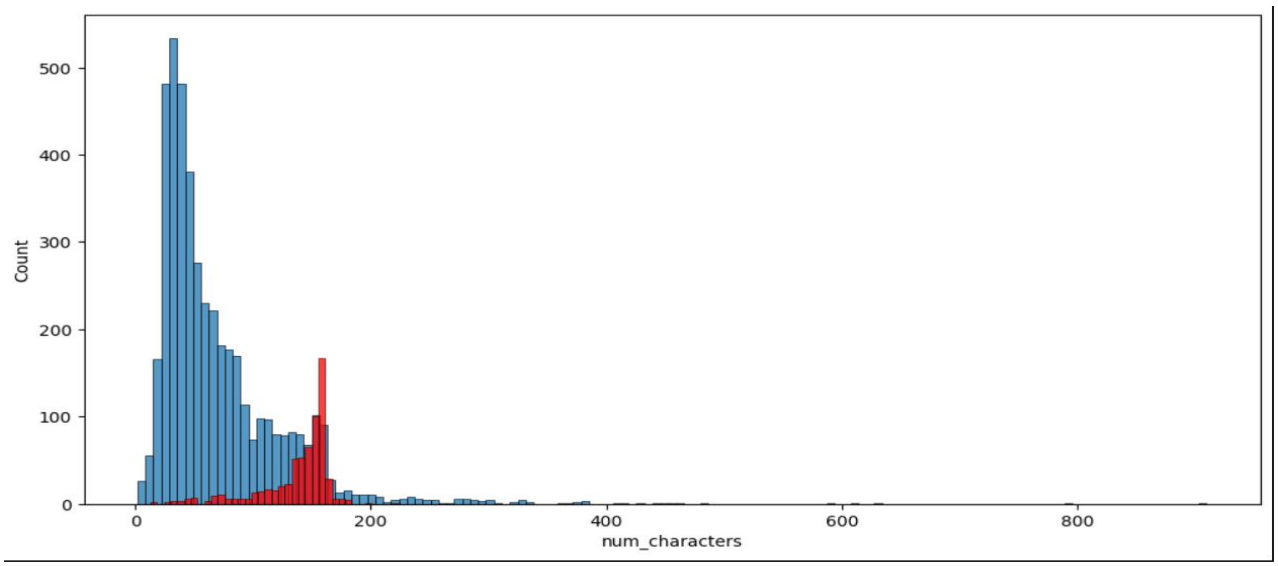

Figure 8: Histogram comparing the number of characters in ham (blue) and spam (red) SMS messages

.

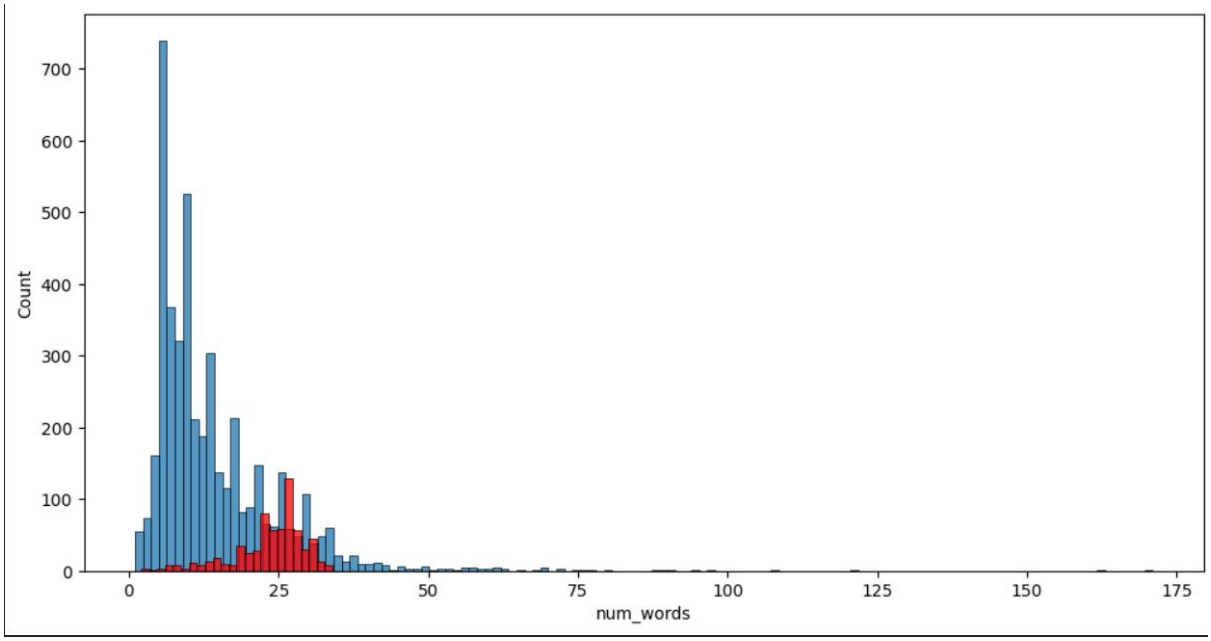

Figure 9: Histogram comparing the number of words in ham (blue) and spam (red) SMS messages

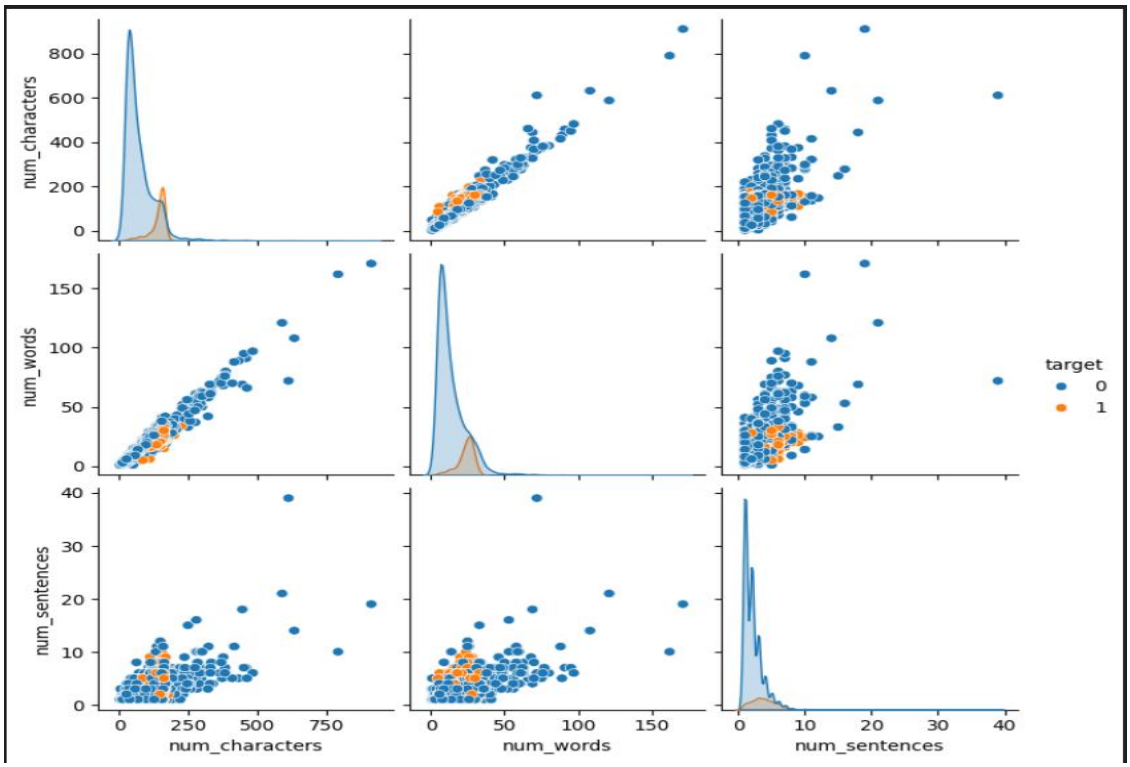

Figure 10: Pairplot illustrating relationships between numerical features

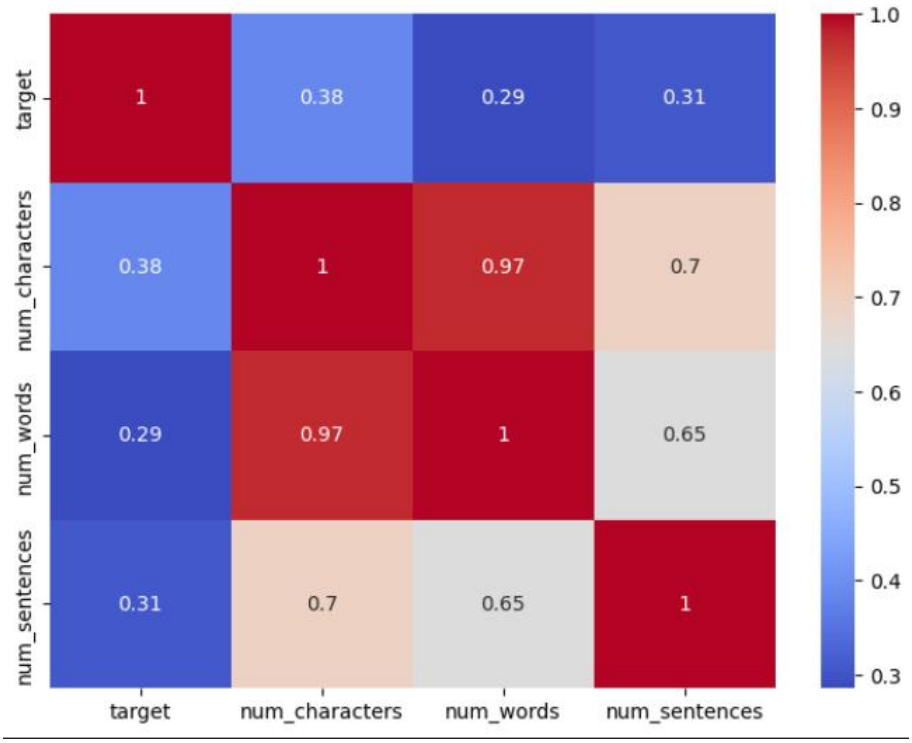

Figure 11: Heatmap of correlations among features

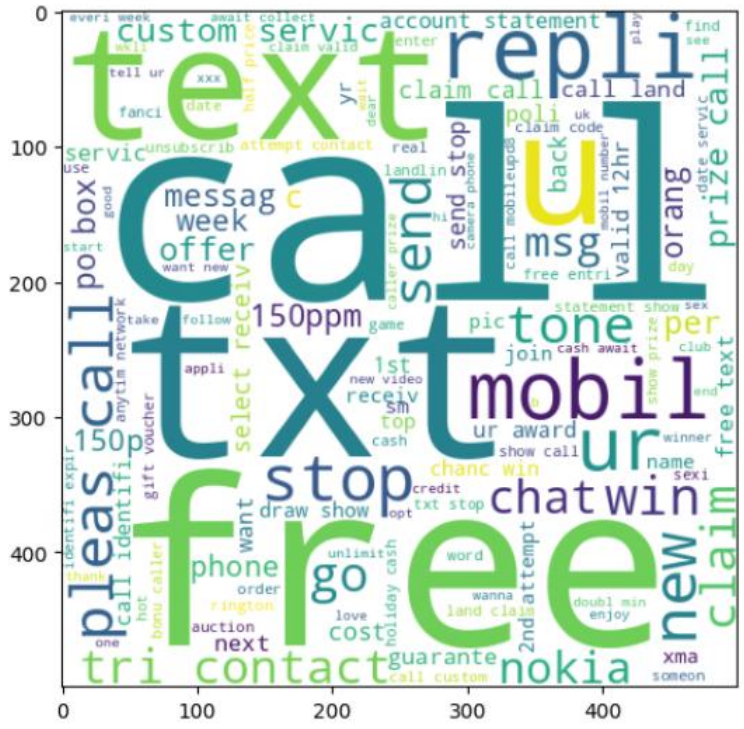

Figure 12: Word cloud displaying the most frequent words found in spam messages

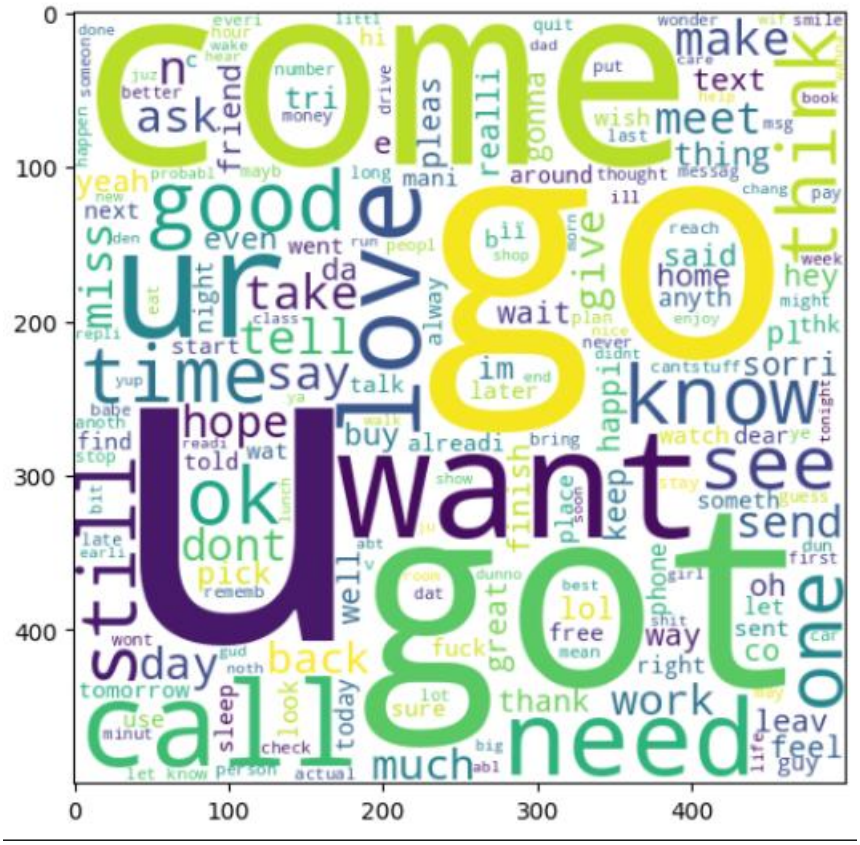

Figure 13: Word cloud displaying the most frequent words found in ham messages

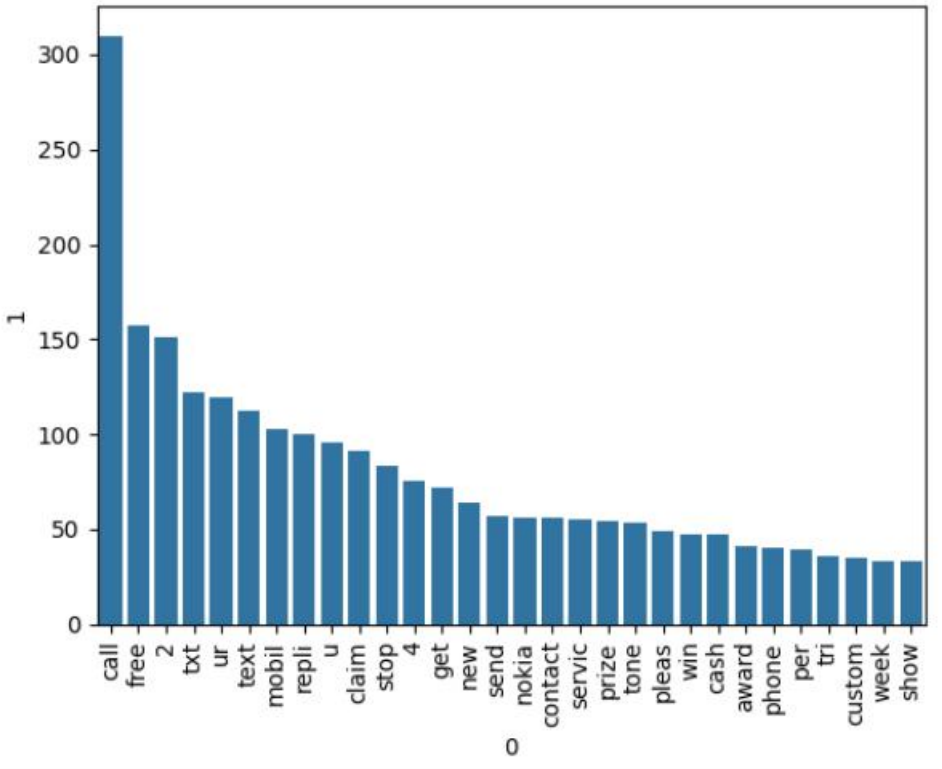

Figure 14: Top 30 most frequent words appearing in spam messages

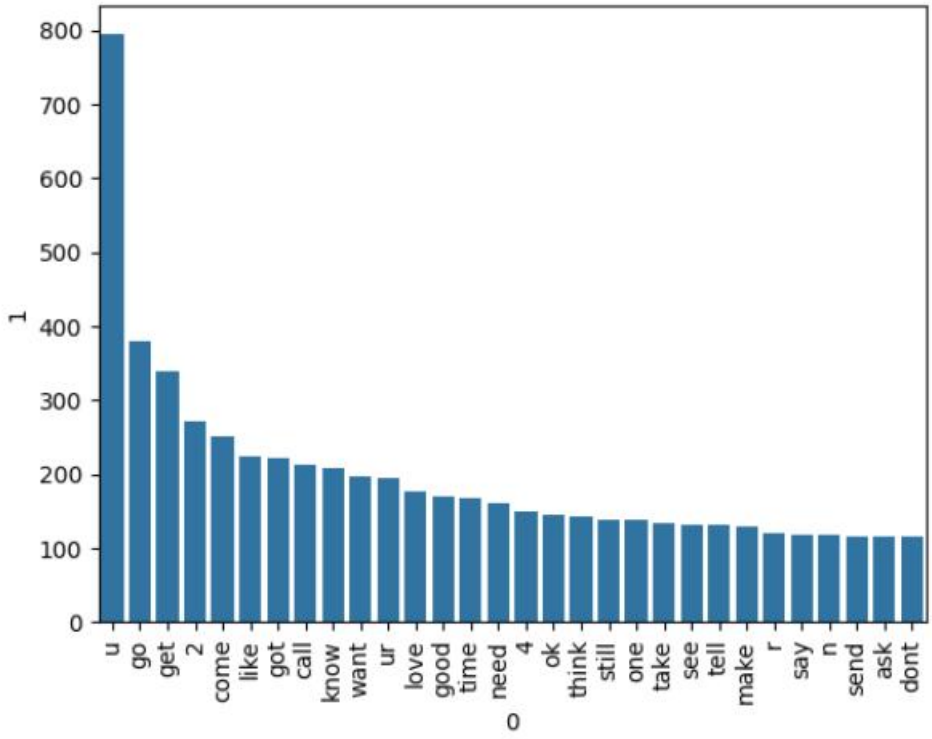

Figure 15: Top 30 most frequent words appearing in ham messages

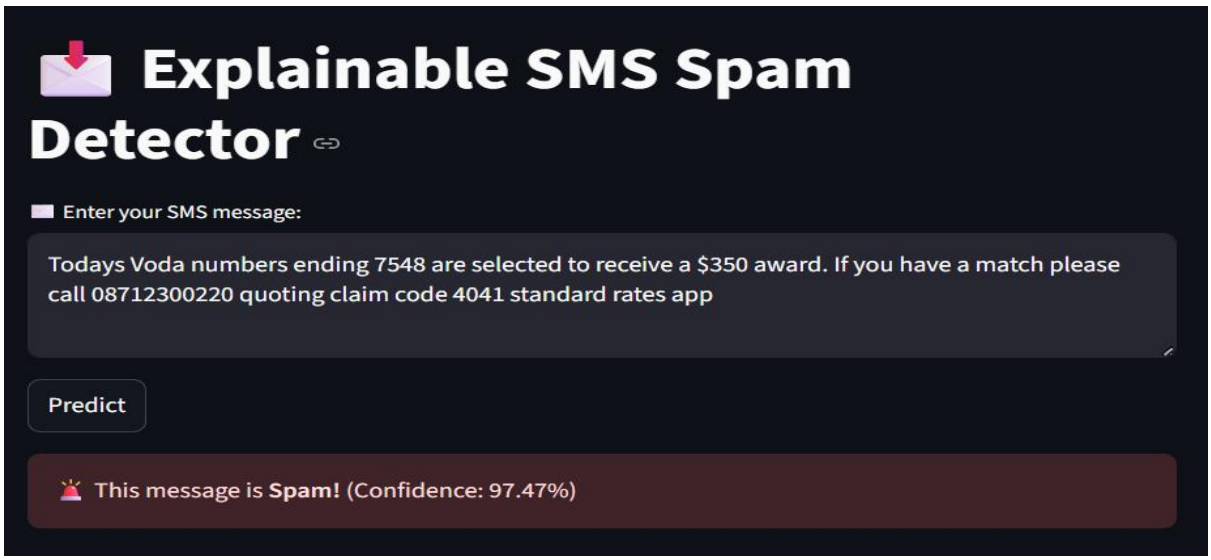

Figure 16: UI for Prediction

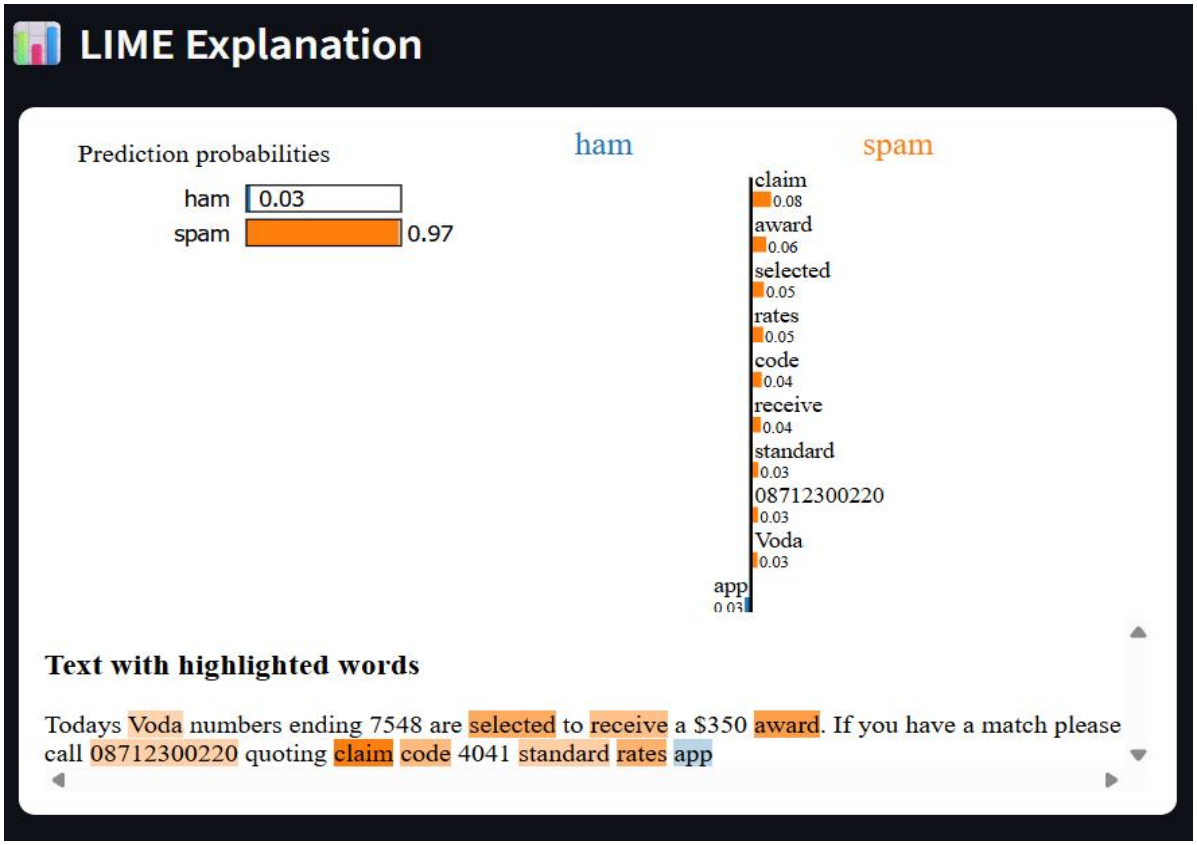

Figure 17: Bar plot for LIME's Local Explanation

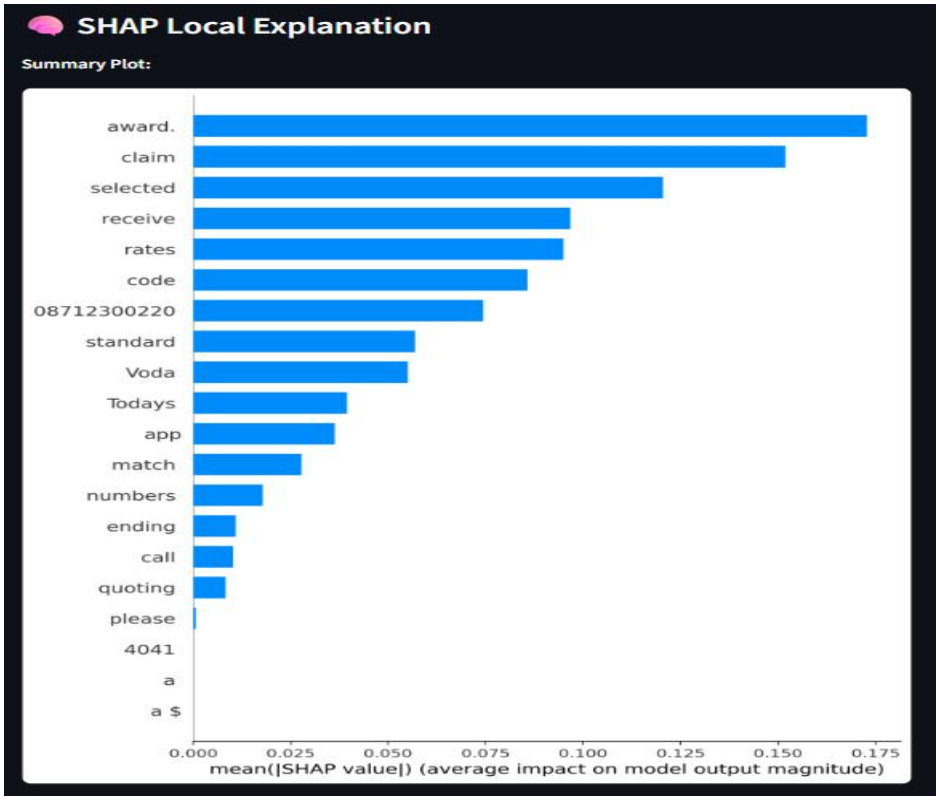

Figure 18: Summary plot for SHAP's Local Explanation

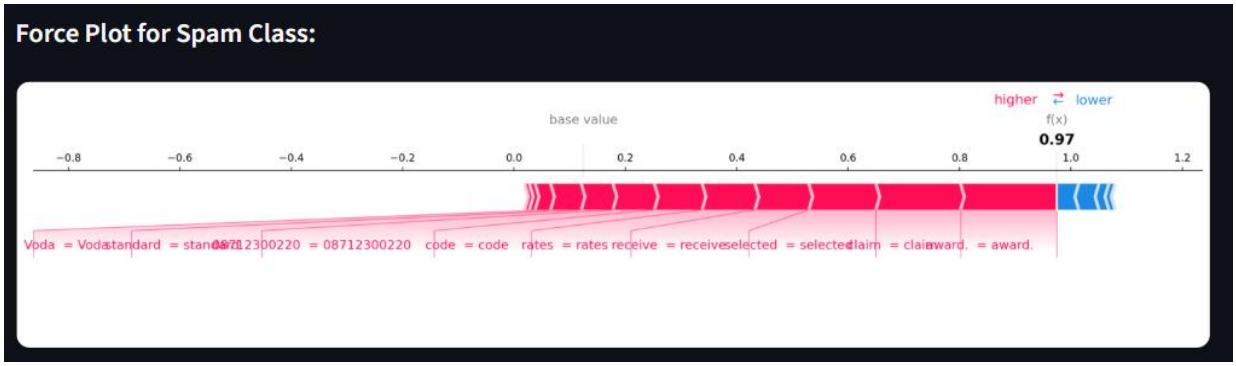

Figure 19: Force plot for SHAP's Local Explanation

Figure 20: Waterfall plot for SHAP's Local Explanation

Hybrid Explanation (LIME + SHAP)

Detailed Method Comparison:

	Feature	Impact Type	LIME Weight	SHAP Weight	Hybrid Weight
0	award	SPAM Indicator	0.0625	0.1728	0.1177
1	claim	SPAM Indicator	0.0822	0.1520	0.1171
2	selected	SPAM Indicator	0.0532	0.1205	0.0868
3	rates	SPAM Indicator	0.0497	0.0950	0.0723
4	receive	SPAM Indicator	0.0403	0.0967	0.0685
5	code	SPAM Indicator	0.0441	0.0858	0.0649
6	08712300220	SPAM Indicator	0.0287	0.0744	0.0515
7	standard	SPAM Indicator	0.0303	0.0569	0.0436
8	Voda	SPAM Indicator	0.0267	0.0551	0.0409
9	app	HAM Indicator	-0.0254	-0.0363	-0.0308

Figure 21: Hybrid Explanation

Stacked Bar Chart - LIME vs SHAP Contributions:

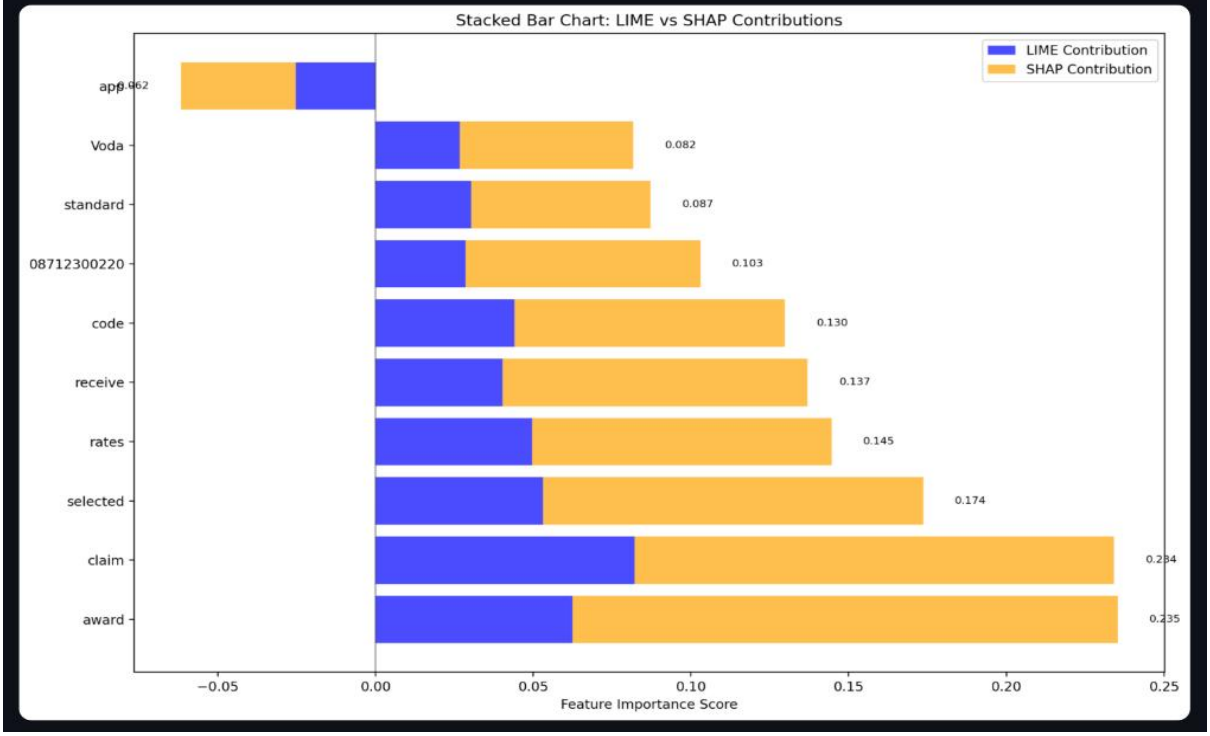

Figure 22: Stack Bar diagram for hybrid explanation

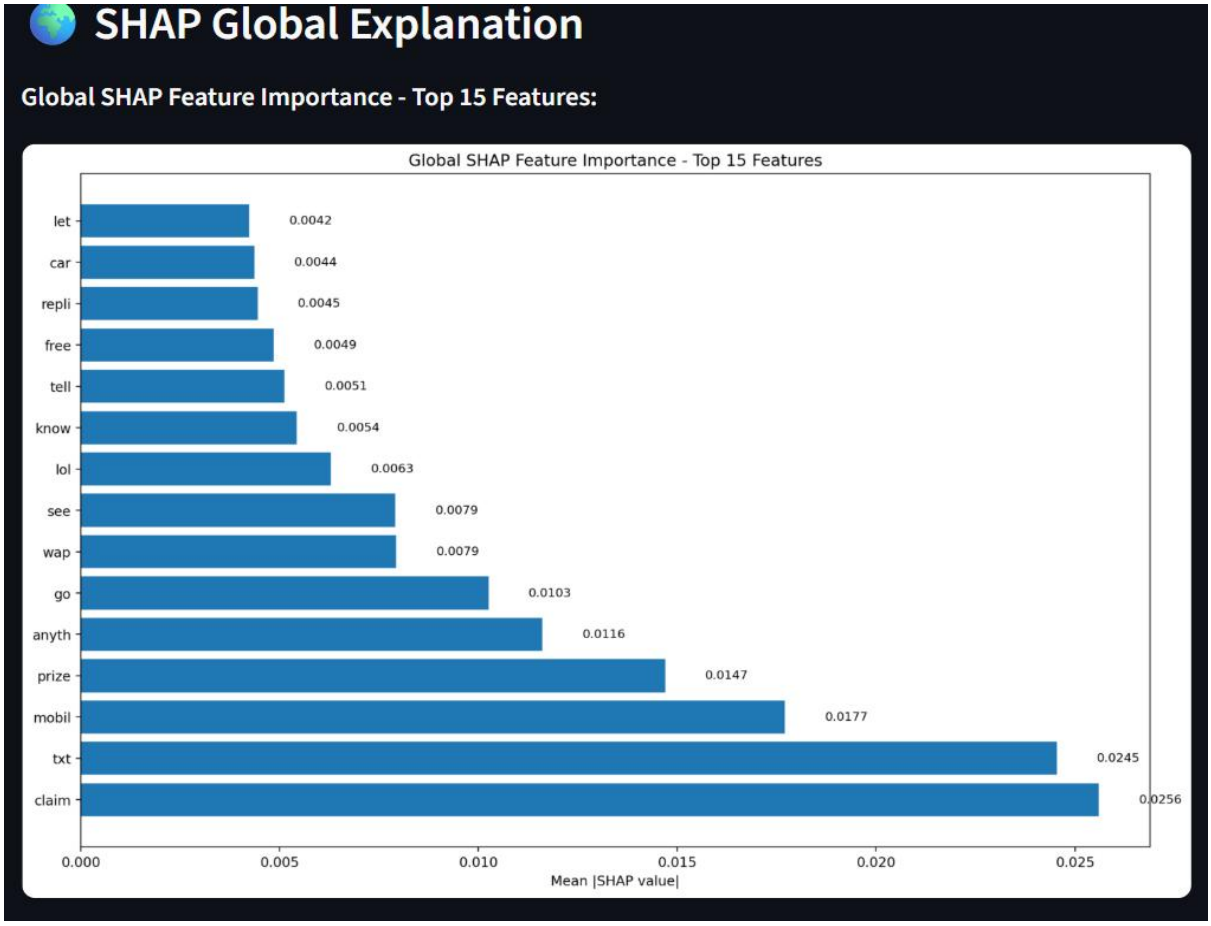

Figure 23: Summary plot for SHAP's global explanation

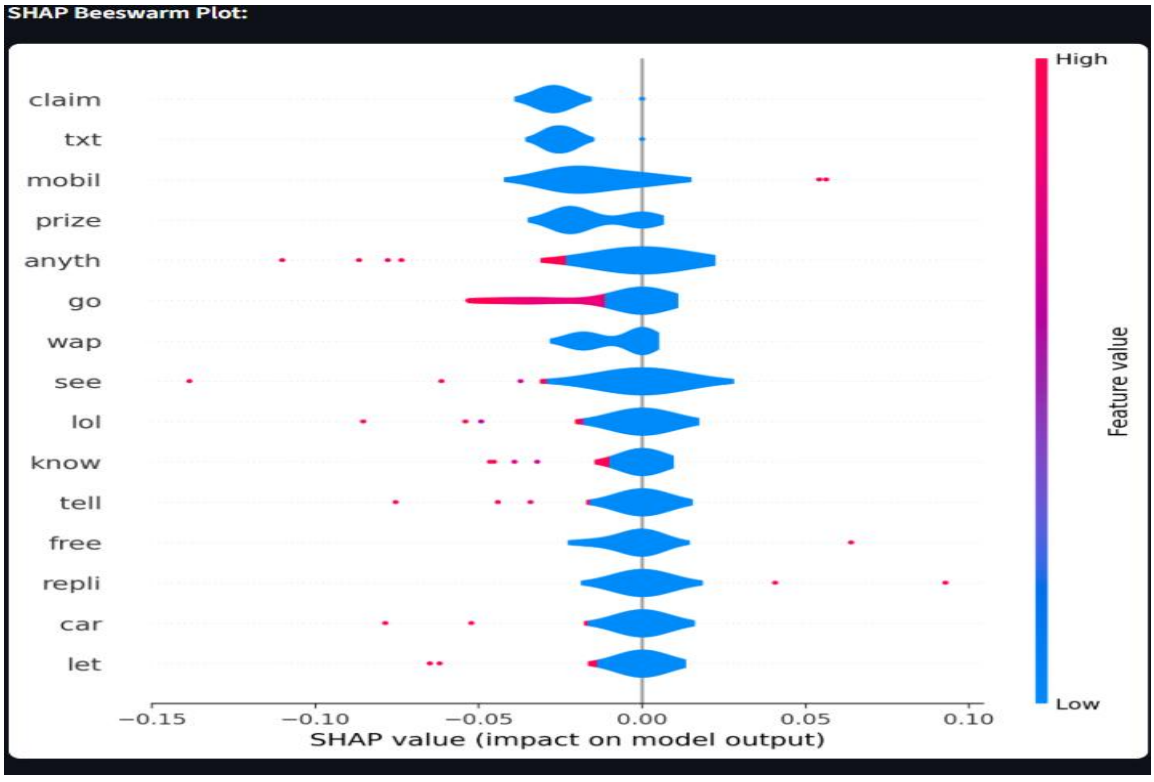

Figure 24: BeeSwarm plot for SHAP's global explanation

Class	Precision	Recall	F1 Score	Support
Ham	0.960000	1.000000	0.980000	896
Spam	0.980000	0.740000	0.840000	138
Accuracy	-	-	0.960000	1034
Macro Avg	0.970000	0.870000	0.910000	1034
Weighted Avg	0.960000	0.960000	0.960000	1034

Figure 25: Classification report of Multinomial Naive Bayes model